

cl-coingecko

A Library for the CoinGecko API

`https:
//github.com/cgore/cl-coingecko/`

Christopher Mark Gore

`http://www.cgore.com`

`cgore@cgore.com`

Monday, August 17th, AD 2026

Contents

Copyright	vii
Introduction	ix
1 The coingecko/rest-api Package	1
1.1 Variables	1
1.1.1 The <code>*v3-public*</code> Variable	1
1.1.2 The <code>*v3-pro*</code> Variable	1
1.1.3 The <code>*api*</code> Variable	1
1.1.4 The <code>*api-plan*</code> Variable	2
1.1.5 The <code>*api-key*</code> Variable	2
1.1.6 The <code>*api-cache-timeout-seconds*</code> Variable	2
1.1.7 The <code>*max-retries*</code> Variable	2
1.1.8 The <code>*retry-wait-seconds*</code> Variable	2
1.1.9 The <code>*coin-id-aliases*</code> Variable	2
1.2 Configuration	2
1.2.1 The <code>use-keyless</code> Function	2
1.2.2 The <code>use-demo</code> Function	3
1.2.3 The <code>use-pro</code> Function	3
1.2.4 The <code>configure-from-environment</code> Function	3
1.3 Query and Path Helpers	4
1.3.1 The <code>query-value</code> Function	4
1.3.2 The <code>query</code> Function	4
1.3.3 The <code>api-path</code> Function	4
1.3.4 The <code>canonicalize-coin-id</code> Function	5
1.3.5 The <code>canonicalize-ids</code> Function	5
1.4 HTTP	5
1.4.1 The <code>http-get</code> Function	5
1.4.2 The <code>http-get-json</code> Function	6
1.4.3 The <code>http-get-cached</code> Function	6
1.4.4 The <code>http-get-json-cached</code> Function	6
1.4.5 The <code>http-get-json-do-not-cache</code> Function	6
1.4.6 The <code>api-get</code> Function	6
1.5 Prices and Histories	7

1.5.1	The ping Function	7
1.5.2	The get-simple-price Function	7
1.5.3	The get-token-price Function	7
1.5.4	The get-supported-vs-currencies Function	7
1.5.5	The get-coin-market-chart Function	8
1.5.6	The get-coin-market-chart-range Function	8
1.5.7	The get-coin-ohlcv Function	8
1.5.8	The get-coin-ohlcv-range Function	8
1.5.9	The get-coin-history Function	8
1.5.10	The get-coins-markets Function	9
1.5.11	The get-coin Function	9
1.5.12	The get-coins-list Function	9
1.6	The Rest of the Surface	9
1.7	Onchain	10
2	The coingecko Package	13
2.1	Time	13
2.1.1	The unix-epoch Parameter	13
2.1.2	The unix->universal Function	13
2.1.3	The universal->unix Function	14
2.1.4	The canonicalize-date Function	14
2.1.5	The canonicalize-unix Function	14
2.2	Current Quotes	14
2.2.1	The quotes Function	14
2.2.2	The price Function	14
2.2.3	The btc/usd Function	15
2.2.4	The eth/usd Function	15
2.2.5	The btc/xag Function	15
2.2.6	The btc/xau Function	15
2.2.7	The eth/xag Function	15
2.2.8	The eth/xau Function	15
2.2.9	The xag/usd Function	15
2.2.10	The xau/usd Function	15
2.2.11	The /btc Function	15
2.2.12	The /eth Function	16
2.2.13	The /xag Function	16
2.2.14	The /xau Function	16
2.2.15	The btc/eth Function	16
2.2.16	The eth/btc Function	16
2.2.17	The fiat-rate Function	16
2.2.18	The 1/fiat-rate Function	16
2.3	Price Histories	16
2.3.1	The market-chart Function	17
2.3.2	The market-chart-range Function	17
2.3.3	The ohlcv Function	17
2.3.4	The ohlcv-range Function	17

2.3.5 The history Function 17

2.3.6 The price-history Function 17

2.3.7 The ohlc-history Function 18

Copyright

Copyright © 2023 – 2026, Christopher Mark Gore,

Soli Deo Gloria,

All rights reserved.

22 Forest Glade Court, Saint Charles, Missouri 63304 USA.

Web: <http://www.cgore.com>

Email: cgore@cgore.com

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
- Neither the name of Christopher Mark Gore nor the names of other contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS “AS IS” AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Introduction

The `cl-coingecko` library is a Common Lisp client for the CoinGecko API [2]. It is *not* an official CoinGecko product; it is an independent wrapper around their public REST surface.

The thin layer lives in the `coingecko/rest-api` package: one function per documented GET path, plus shared authentication, query encoding, and a sixty-second `function-cache`. The `coingecko` package re-exports that layer and adds the helpers I actually call from <https://limbic.fi> — current quotes and price histories for Bitcoin and friends, including gold (`xau`) and silver (`xag`).

The library covers the v3 REST endpoints from CoinGecko's OpenAPI specification [1]: simple prices, coin metadata, historical charts and OHLC, exchanges, derivatives, treasuries, NFTs, search, global stats, and the `onchain` / `GeckoTerminal` paths. WebSockets and webhooks are not wrapped.

You can download the library from GitHub at:
<https://github.com/cgore/cl-coingecko>

Getting Lisp

Before using this library you need a working Lisp. I use and recommend SBCL, Steel Bank Common Lisp, which is available at:

<http://www.sbcl.org>

SBCL has information on getting started at:

<http://www.sbcl.org/getting.html>

If you are using Debian or a similar Linux distribution (including Ubuntu), you can just run as root:

```
apt-get install sbcl sbcl-doc sbcl-source
```

Getting EMACS and SLIME

After installing, the best way to interact with any Common Lisp is via SLIME, the Superior Lisp Interaction Mode for EMACS, which is available at:

<http://common-lisp.net/project/slime/>

This can be installed on Debian by:

```
apt-get install slime emacs emacs-goodies-el
```

Using the Library

Clone the repository next to your other local systems, or symlink the

ASDF file into a directory on asdf:`*central-registry*` (I use `~/programming/lisp/system`)

```
mkdir -p ~/programming/lisp
cd ~/programming/lisp
git clone git@github.com:cgore/cl-coingecko.git
ln -s ~/programming/lisp/cl-coingecko/coingecko.asd \
    ~/programming/lisp/systems/
```

The system depends on `alexandria`, `dexador`, `function-cache`, `quri`, `sigma`, and `yason`. Those come from `Quicklisp`.

```
(ql:quickload :coingecko)
(in-package :coingecko)
(price-history :bitcoin :vs-currency :usd :days 365)
(btc/usd)
```

Authentication

CoinGecko currently has three REST plans. The library picks one at load time from the process environment, or you can switch in the image.

Plan	Host	Header
Keyless	public v3	none
Demo	public v3	x-cg-demo-api-key
Pro	Pro v3	x-cg-pro-api-key

The public host is `https://api.coingecko.com/api/v3`. The Pro host is `https://pro-api.coingecko.com/api/v3`.

On load it reads, in order, `COINGECKO_PRO_API_KEY`, `COINGECKO_DEMO_API_KEY`, then `COINGECKO_API_KEY`. You can also call `use-demo`, `use-pro`, or `use-keyless`.

Keyless is fine for trying things. It is rate-limited (roughly 10–30 calls per minute, shared by IP) and is not suitable for production polling. Get a Demo or Pro key from the CoinGecko developer dashboard.

Some endpoints — OHLC range, new listings, news, most onchain filters, enterprise supply charts — require a paid plan. The wrappers are still there; CoinGecko will reject them if your key cannot call them.

Testing

Specs are `sigma/behavior` / `should forms`, the same style as Σ . Pure helpers run at load time. `asdf:test-system` reloads the sources so those assertions run again.

```
(asdf:test-system :coingecko)
```

To also hit the live API (ping, a Bitcoin price, and a one-day history), set `COINGECKO_LIVE_TESTS=1` in the environment first. A Demo key is enough; keyless works if you have not already exhausted the public rate limit.

Have fun!

Chapter 1

The coingecko/rest-api Package

The `coingecko/rest-api` package is the thin HTTP layer. Every documented CoinGecko v3 GET lives here as a function whose keyword arguments are the query parameters with underscores turned into hyphens. Lists become comma-separated. `t` / `:true` / `:false` become `true/false`. Results are `yason` hash tables, with JSON arrays parsed as vectors and booleans as symbols.

Onchain / GeckoTerminal endpoints live in the same package, in `source/onchain.lisp`. They are listed at the end of this chapter.

1.1 Variables

1.1.1 The `*v3-public*` Variable

The public (Demo / keyless) API root.
`https://api.coingecko.com/api/v3`

1.1.2 The `*v3-pro*` Variable

The Pro API root.
`https://pro-api.coingecko.com/api/v3`

1.1.3 The `*api*` Variable

The host actually used for requests. `use-keyless` and `use-demo` set this to `*v3-public*`; `use-pro` sets it to `*v3-pro*`.

1.1.4 The `*api-plan*` Variable

One of `:keyless`, `:demo`, or `:pro`. This controls both the host and whether an API-key header is sent.

1.1.5 The `*api-key*` Variable

The Demo or Pro key, or `nil`. Loaded from the environment by `configure-from-environment` when the system is loaded.

1.1.6 The `*api-cache-timeout-seconds*` Variable

How long `http-get-cached` and `http-get-json-cached` keep a response. The default is 60, which matches CoinGecko's public `/simple/price` cache.

1.1.7 The `*max-retries*` Variable

How many times to retry a request that comes back 429. Default 3.

1.1.8 The `*retry-wait-seconds*` Variable

Base wait, in seconds, between 429 retries. Each attempt waits `attempt × *retry-wait-seconds*`. Default 2.

1.1.9 The `*coin-id-aliases*` Variable

An equal-hash of short tickers to CoinGecko coin ids: `"btc" → "bitcoin"`, `"eth" → "ethereum"`, and a handful of other majors. `canonicalize-coin-id` consults this table.

1.2 Configuration

1.2.1 The `use-keyless` Function

Talk to the public host with no API-key header.

Syntax

```
(use-keyless)
```

Returns

```
:keyless.
```

1.2.2 The use-demo Function

Talk to the public host with an `x-cg-demo-api-key` header.

Syntax

```
(use-demo &optional key)
```

Arguments and Values

key If supplied, stored in `*api-key*`.

Returns

`:demo`.

1.2.3 The use-pro Function

Talk to the Pro host with an `x-cg-pro-api-key` header.

Syntax

```
(use-pro &optional key)
```

Arguments and Values

key If supplied, stored in `*api-key*`.

Returns

`:pro`.

1.2.4 The configure-from-environment Function

Pick `:pro`, `:demo`, or `:keyless` from the process environment. Called automatically when `source/rest-api.lisp` is loaded.

Syntax

```
(configure-from-environment)
```

Affected By

`COINGECKO_PRO_API_KEY`, `COINGECKO_DEMO_API_KEY`, `COINGECKO_API_KEY`.

1.3 Query and Path Helpers

1.3.1 The query-value Function

Turn a Lisp value into a CoinGecko query-string fragment.

Syntax

```
(query-value value)
```

Arguments and Values

value *t* or `:true` become `"true"`; `:false` becomes `"false"`; `nil` becomes `nil` (the pair is dropped); a list or vector becomes comma-separated; a symbol is downcased; a string or number is passed through.

Examples

```
(query-value :usd)           ; => "usd"
(query-value '(:btc :eth))    ; => "btc,eth"
(query-value t)               ; => "true"
(query-value nil)             ; => NIL
```

1.3.2 The query Function

Build a list of (name value) pairs from alternating names and values, dropping nils. Pairs are proper lists so function-cache can hash them; `http-get` turns them into an alist for `quri`.

Syntax

```
(query &rest pairs)
```

Examples

```
(query "vs_currency" :usd "days" 30 "interval" nil)
; => (("vs_currency" "usd") ("days" "30"))
```

1.3.3 The api-path Function

Join URL segments onto the API root path.

Syntax

```
(api-path &rest segments)
```

Examples

```
(api-path "coins" :bitcoin "market_chart")
; => "/coins/bitcoin/market_chart"
```

1.3.4 The canonicalize-coin-id Function

Downcase a coin id. Known ticker symbols such as `:btc` map to CoinGecko ids via `*coin-id-aliases*`. Unknown strings pass through downcased, so a real CoinGecko id such as `"wrapped-bitcoin"` is left alone.

Syntax

```
(canonicalize-coin-id id)
```

Examples

```
(canonicalize-coin-id :btc)           ; => "bitcoin"
(canonicalize-coin-id "BTC")         ; => "bitcoin"
(canonicalize-coin-id "wrapped-bitcoin") ; => "wrapped-bitcoin"
```

1.3.5 The canonicalize-ids Function

Canonicalize a single coin id or a list or vector of them.

1.4 HTTP**1.4.1 The http-get Function**

Make an HTTP GET to the CoinGecko API. Retries 429s with linear backoff up to `*max-retries*`.

Syntax

```
(http-get url-components &key (query-args ' ()))
```

Arguments and Values

url-components A string starting with `/`, or a list of strings to concatenate after `*api*`.

query-args Query pairs as produced by `query`.

Returns

The response body as a string.

1.4.2 The `http-get-json` Function

Like `http-get`, but parse the body as JSON with `yason`. Arrays become vectors; booleans become symbols.

1.4.3 The `http-get-cached` Function

`function-cache` wrapper around `http-get`, `timeout` `*api-cache-timeout-seconds*`.

1.4.4 The `http-get-json-cached` Function

`function-cache` wrapper around `http-get-json`.

1.4.5 The `http-get-json-do-not-cache` Function

An alias for `http-get-json`, so call sites can say they mean it when they skip the cache.

1.4.6 The `api-get` Function

The function every endpoint wrapper calls.

Syntax

```
(api-get path &key query (cache t))
```

Arguments and Values

path A path already rooted at `/`, as from `api-path`.

query Query pairs from `query`, or `nil`.

cache When true (the default), use `http-get-json-cached`; otherwise `http-get-json`.

1.5 Prices and Histories

These are the endpoints Limbic actually needs. Every wrapper also takes `&key (cache t)` so you can bypass the sixty-second cache.

1.5.1 The ping Function

GET `/ping` — check that the API is up.

Syntax

```
(ping &key (cache t))
```

Returns

A hash table; the interesting key is `"gecko_says"`.

1.5.2 The get-simple-price Function

GET `/simple/price` — current prices by CoinGecko id, name, or symbol. The `ids` argument is run through `canonicalize-ids`, so `:btc` works.

Syntax

```
(get-simple-price &key vs-currencies ids names symbols include-tokens  
include-market-cap include-24hr-vol include-24hr-change include-last-updated-at  
precision (cache t))
```

Examples

```
(get-simple-price :ids '(:btc :eth)  
                  :vs-currencies '(:usd :xag :xau)  
                  :include-24hr-change t)
```

1.5.3 The get-token-price Function

GET `/simple/token_price/{id}` — prices by contract address on an asset platform. Here `id` is the platform (`:ethereum`), not a coin.

1.5.4 The get-supported-vs-currencies Function

GET `/simple/supported_vs_currencies` — the legal values for `vs-currency` / `vs-currencies`, including `usd`, `xag`, and `xau`.

1.5.5 The `get-coin-market-chart` Function

GET `/coins/{id}/market_chart` — historical prices, market_caps, and total_volumes as vectors of [unix-ms, value] pairs.

Syntax

```
(get-coin-market-chart id &key vs-currency days interval precision
 (cache t))
```

Arguments and Values

id A CoinGecko coin id, or a known ticker such as `:btc`.

vs-currency Target currency, e.g. `:usd`.

days An integer or "max".

interval nil for auto-granularity, or `:daily`, `:hourly`, or `:5m`.

CoinGecko's auto-granularity is: 1 day from now is 5-minute points, 2–90 days are hourly, longer is daily (00:00 UTC).

Examples

```
(get-coin-market-chart :bitcoin :vs-currency :usd :days 365
                        :interval :daily)
```

1.5.6 The `get-coin-market-chart-range` Function

GET `/coins/{id}/market_chart/range` — the same series between UNIX *from* and *to*.

1.5.7 The `get-coin-ohlcv` Function

GET `/coins/{id}/ohlcv` — candles as [timestamp, open, high, low, close]. *days* is one of 1, 7, 14, 30, 90, 180, 365, or max.

1.5.8 The `get-coin-ohlcv-range` Function

GET `/coins/{id}/ohlcv/range`. Analyst plan and above.

1.5.9 The `get-coin-history` Function

GET `/coins/{id}/history` — a snapshot at 00:00 UTC on *date*. CoinGecko wants YYYY-MM-DD.

1.5.10 The get-coins-markets Function

GET /coins/markets — a page of coins with price, market cap, volume, and related market data. `ids` is canonicalized.

1.5.11 The get-coin Function

GET /coins/{id} — metadata and market data for one coin.

1.5.12 The get-coins-list Function

GET /coins/list — every supported coin's id, name, and symbol.

1.6 The Rest of the Surface

The remaining wrappers follow the same pattern: path parameters are required arguments, query parameters are keywords, and `:cache` defaults to true (`get-key` defaults to false). Analyst- and Enterprise-only routes are still defined; CoinGecko will reject them if your plan does not include them.

Function	Path
get-key	/key
get-coins-list-new	/coins/list/new
get-coin-tickers	/coins/{id}/tickers
get-coin-by-contract	/coins/{id}/contract/{contract_address}
get-coin-market-chart-by-contract	/coins/{id}/contract/{contract_address}/market_chart
get-coin-market-chart-range-by-contract	/coins/{id}/contract/{contract_address}/market_chart/range
get-coin-circulating-supply-chart	/coins/{id}/circulating_supply_chart
get-coin-circulating-supply-chart-range	/coins/{id}/circulating_supply_chart/range
get-coin-total-supply-chart	/coins/{id}/total_supply_chart
get-coin-total-supply-chart-range	/coins/{id}/total_supply_chart/range
get-coin-supply-breakdown	/coins/{id}/supply_breakdown
get-top-gainers-losers	/coins/top_gainers_losers
get-coins-categories-list	/coins/categories/list
get-coins-categories	/coins/categories
get-asset-platforms	/asset_platforms
get-token-list	/token_lists/{asset_platform_id}/all.json
get-exchanges	/exchanges
get-exchanges-list	/exchanges/list
get-exchange	/exchanges/{id}
get-exchange-tickers	/exchanges/{id}/tickers
get-exchange-volume-chart	/exchanges/{id}/volume_chart
get-exchange-volume-chart-range	/exchanges/{id}/volume_chart/range
get-derivatives	/derivatives
get-derivatives-exchanges	/derivatives/exchanges
get-derivatives-exchange	/derivatives/exchanges/{id}

Function	Path
get-derivatives-exchanges-list	/derivatives/exchanges/list
get-entities-list	/entities/list
get-public-treasury-by-coin	/public.treasury/{coin_id}
get-public-treasury	/public.treasury/{entity_id}
get-public-treasury-holding-chart	/public.treasury/{coin_id}/holding.chart
get-public-treasury-transaction-history	/public.treasury/{coin_id}/transaction.history
get-nfts-list	/nfts/list
get-nft	/nfts/{id}
get-nft-by-contract	/nfts/{platform}/contract/{address}
get-nfts-markets	/nfts/markets
get-nft-market-chart	/nfts/{id}/market.chart
get-nft-market-chart-by-contract	/nfts/{platform}/contract/{address}/market.chart
get-nft-tickers	/nfts/{id}/tickers
get-search	/search
get-trending	/search/trending
get-news	/news
get-insights	/insights
get-global	/global
get-global-defi	/global/decentralized.finance.defi
get-global-market-cap-chart	/global/market.cap.chart
get-exchange-rates	/exchange.rates

See `coingecko.restclient` in the repository root for curl-style examples of the same paths.

1.7 Onchain

These functions are defined in `source/onchain.lisp` but exported from `coingecko/rest-api`. They talk to the same v3 host when you have a key. Path parameters that are addresses may be a string or a list (joined with commas).

Function	Path
get-onchain-token-price	/onchain/simple/networks/.../token.price/...
get-onchain-pool	/onchain/networks/{network}/pools/{address}
get-onchain-pools	.../pools/multi/{addresses}
get-onchain-pool-info	.../pools/{pool.address}/info
get-onchain-top-pools	/onchain/networks/{network}/pools
get-onchain-top-pools-by-dex	.../dexes/{dex}/pools
get-onchain-top-pools-by-token	.../tokens/{token}/pools
get-onchain-pools-megafilter	/onchain/pools/megafilter
get-onchain-new-pools	/onchain/networks/new.pools
get-onchain-new-pools-by-network	.../{network}/new.pools
get-onchain-trending-pools	/onchain/networks/trending.pools
get-onchain-trending-pools-by-network	.../{network}/trending.pools

Function	Path
get-onchain-trending-search-pools	/onchain/pools/trending_search
get-onchain-token	/onchain/networks/{network}/tokens/{address}
get-onchain-tokens	.../tokens/multi/{addresses}
get-onchain-token-info	.../tokens/{address}/info
get-onchain-tokens-recently-updated	/onchain/tokens/info_recently_updated
get-onchain-pool-ohlcv	.../pools/.../ohlcv/{timeframe}
get-onchain-token-ohlcv	.../tokens/.../ohlcv/{timeframe}
get-onchain-pool-trades	.../pools/.../trades
get-onchain-token-trades	.../tokens/.../trades
get-onchain-top-traders	.../tokens/.../top_traders
get-onchain-top-holders	.../tokens/.../top_holders
get-onchain-holders-chart	.../tokens/.../holders_chart
get-onchain-categories	/onchain/categories
get-onchain-category-pools	/onchain/categories/{id}/pools
get-onchain-networks	/onchain/networks
get-onchain-dexes	/onchain/networks/{network}/dexes
get-onchain-search-pools	/onchain/search/pools

Chapter 2

The coingecko Package

The `coingecko` package is the one you use from application code. It re-exports every external symbol of `coingecko/rest-api` and adds the helpers I call from `Limbic`: current quotes, price histories as Common Lisp universal times, and the old Zapper-era names such as `btc/usd` and `xag/usd`.

2.1 Time

CoinGecko chart points are UNIX milliseconds. Common Lisp wants universal time. These four functions sit between the two.

2.1.1 The `unix-epoch` Parameter

Universal time of the UNIX epoch, 1970-01-01T00:00:00Z.

2.1.2 The `unix->universal` Function

Convert a UNIX timestamp in seconds or milliseconds to universal time. Values above 10^{11} are treated as milliseconds, which is what CoinGecko chart points use.

Syntax

```
(unix->universal unix)
```

Examples

```
(unix->universal 0)           ; => unix-epoch  
(unix->universal 1577836800)  ; => 2020-01-01T00:00:00Z  
(unix->universal 1577836800000) ; => same instant, from milliseconds
```

2.1.3 The universal->unix Function

Convert a Common Lisp universal time to UNIX seconds.

2.1.4 The canonicalize-date Function

CoinGecko /history dates are YYYY-MM-DD. This accepts a string (passed through), a universal time, or a (year month day) list.

Syntax

```
(canonicalize-date date)
```

Examples

```
(canonicalize-date "2024-01-15")    ; => "2024-01-15"
(canonicalize-date '(2024 7 4))    ; => "2024-07-04"
```

2.1.5 The canonicalize-unix Function

Turn a value into a UNIX-seconds string for range endpoints. Integers greater than 10^{11} are treated as milliseconds; integers greater than $3 \cdot 10^9$ are treated as universal times; everything else is passed through as UNIX seconds.

2.2 Current Quotes

2.2.1 The quotes Function

One cached /simple/price call covering the usual Limbic quote set: Bitcoin and Ether versus usd, xag, and xau.

Syntax

```
(quotes &key (ids '(:bitcoin :ethereum)) (vs-currencies '(:usd
:xag :xau)))
```

2.2.2 The price Function

Current price of a coin in a vs-currency. The id may be :btc or "bitcoin".

Syntax

```
(price id &optional (vs-currency :usd))
```

Examples

```
(price :bitcoin :usd)
(price :eth :usd)
```

2.2.3 The btc/usd Function

The value of Bitcoin in US Dollars.

2.2.4 The eth/usd Function

The value of Ether in US Dollars.

2.2.5 The btc/xag Function

The value of Bitcoin in troy ounces of silver.

2.2.6 The btc/xau Function

The value of Bitcoin in troy ounces of gold.

2.2.7 The eth/xag Function

The value of Ether in troy ounces of silver.

2.2.8 The eth/xau Function

The value of Ether in troy ounces of gold.

2.2.9 The xag/usd Function

The value of one troy ounce of silver in US Dollars, derived as $(\text{btc/usd}) / (\text{btc/xag})$.

2.2.10 The xau/usd Function

The value of one troy ounce of gold in US Dollars, derived as $(\text{btc/usd}) / (\text{btc/xau})$.

2.2.11 The /btc Function

The value of *id* expressed in Bitcoin, via US Dollars.

Syntax

```
(/btc id)
```

2.2.12 The /eth Function

The value of *id* expressed in Ether.

2.2.13 The /xag Function

The value of *id* expressed in troy ounces of silver.

2.2.14 The /xau Function

The value of *id* expressed in troy ounces of gold.

2.2.15 The btc/eth Function

The value of Bitcoin expressed in Ether.

2.2.16 The eth/btc Function

The value of Ether expressed in Bitcoin.

2.2.17 The fiat-rate Function

USD-per-unit for a vs-currency (:usd, :xag, :xau) or a coin (:btc, :eth). The name is the Zapper-era one, so Limbic call sites can switch over without renaming.

Syntax

```
(fiat-rate symbol)
```

2.2.18 The 1/fiat-rate Function

Units per US Dollar, the inverse of `fiat-rate`.

2.3 Price Histories

This is the main Limbic entry point: Bitcoin (and other) histories as lists of Common Lisp times and prices, rather than raw CoinGecko payloads.

2.3.1 The market-chart Function

Raw /coins/{id}/market_chart payload: a hash table with "prices", "market_caps", and "total_volumes".

Syntax

```
(market-chart id &key (vs-currency :usd) (days 30) interval
precision)
```

2.3.2 The market-chart-range Function

Raw /coins/{id}/market_chart/range payload. *from* and *to* may be universal times, UNIX seconds, or UNIX milliseconds.

2.3.3 The ohlc Function

Raw /coins/{id}/ohlc payload: a vector of [timestamp, open, high, low, close].

2.3.4 The ohlc-range Function

Raw /coins/{id}/ohlc/range payload. Analyst plan and above.

2.3.5 The history Function

Snapshot of a coin at 00:00 UTC on *date*. The date may be YYYY-MM-DD, a universal time, or (year month day).

Syntax

```
(history id date &key localization)
```

Examples

```
(history :bitcoin "2024-01-15")
(history :bitcoin '(2024 1 15))
```

2.3.6 The price-history Function

A list of (universal-time price) points for *id*. This is the function I reach for when I want Bitcoin's history.

Syntax

```
(price-history id &key (vs-currency :usd) (days 30) interval)
```

Examples

```
(price-history :bitcoin :vs-currency :usd :days 365)
(price-history :btc :days 90 :interval :daily)
```

Returns

A list of two-element lists. The first element is a universal time; the second is the price in *vs-currency*.

2.3.7 The ohlc-history Function

A list of (universal-time open high low close) candles for *id*.

Syntax

```
(ohlc-history id &key (vs-currency :usd) (days 30) interval)
```

Examples

```
(ohlc-history :bitcoin :days 30)
```

Bibliography

- [1] COINGECKO. Coingecko API — openAPI specification. <https://github.com/coingecko/coingecko-api-oas>, 2026.
- [2] COINGECKO. Coingecko API documentation. <https://docs.coingecko.com>, 2026.